DESIGNING A MEMORY SYSTEM (RAM + ROM)

Note:

- Use Ubuntu OS since GTKWave is available in Ubuntu.
- Upload your solution on Quanta by 5:50 pm.

This lab aims to design and simulate a **memory system** consisting of a 1 KB RAM and a 1 KB ROM, connected to a processor bus through an address decoder and chip-select logic. The memory is built hierarchically from basic D flip-flops upward.

The RAM uses D flip-flops to store each bit. Every 16 bits are grouped together to form a **16-bit register** (one memory word). The memory uses the **Big-Endian** format — the lower address holds the higher byte and vice versa.

Each **ram32byte** block contains 32 sixteen-bit registers (32 × 2 bytes = 64 bytes). A **5-to-32 decoder** selects which register to write to, and a **32-to-1 mux** selects which register to read from. Thirty-two such blocks are stacked to form **ram1kb** (32 × 64 bytes ≈ 2 KB addressable space, with 10-bit addressing). A structurally identical **rom1kb** module provides read-only storage with pre-loaded contents.

The top-level **memory_system** module connects ram1kb and rom1kb to the 20-bit processor address bus, generating chip-select signals from the upper address bits and routing active-low control signals (RD_n, WR_n, M_IO_n) to the appropriate submodules.

Some important points to note:

1. Since this lab will be auto-evaluated, make sure you follow all implementation details properly and write runnable code.
2. Each module may have partial marks, but since some modules depend on others, follow the order given below.
3. The modules are already declared in their respective files and you only need to complete the logic of the body. Make sure you don't modify the header.
4. You need to open the **impl** folder in VSCode.
5. You can use `python3 mem_test.py` to check your code for correctness after completing each module.
6. After finishing, use `python3 create_submission.py` to create the submission zip and submit it on Quanta. Only submissions that follow this will be evaluated.
7. If you get a permission denied error, use `chmod +x './DO NOT OPEN/dist/mem_test_final'`
8. The **outputs** folder contains the expected waveform outputs for each module.

The implementation is divided into several modules **(please follow the given order, as the modules are dependent on each other):**

---

1. `D_ff(clk, chipSelect, reset, regWrite, enable, d, q)`

a. The D flip-flop must work at the **positive edge of the clock**.

b. Since the user can have multiple chips, a **chipSelect** input signifies whether this chip is selected.

c. If the chip is selected and **reset** is high, the output q should be set to 0. (This should also happen only at the posedge of the clock.)

d. If the chip is selected, the flip-flop is **enabled**, and **regWrite** is high, the input d is written into the output q.

e. Please be aware that there is **no need to make modifications** to the D_ff module. It has been provided to you in its entirety to prevent any fundamental errors.

---

2. `mux2to1_16bits(in0, in1, select, muxOut)` **(1 mark)**

a. This is a **16-bit 2-to-1 multiplexer** that selects `in0` or `in1` as `muxOut`, depending on the `select` input.

b. This mux is used as the **output gate** of each memory block. When `outputEnable` is 0 or `chipSelect` is 0, the combined select signal is 0 and `muxOut` is driven to `16'b0`, releasing the data bus. When both are 1, `muxOut` passes the memory read data.

---

3. `decoder5to32(selectIn, decOut)` **(2 marks)**

a. This decoder takes a **5-bit input** `selectIn[4:0]` and asserts exactly **one** of 32 output lines in `decOut[31:0]`.

b. It is used at two levels: within `ram32byte` to select which of the 32 registers to write to, and within `ram1kb` / `rom1kb` to select which of the 32 sub-blocks is active.

c. When `selectIn = n`, only `decOut[n]` should be 1; all other bits should be 0.

d. The output is **one-hot encoded** — at most one bit is high at any time.

Note: A `decoder4to16(destReg, decOut)` is also used internally (e.g., within the register file stage if needed). Its structure is identical but with a 4-bit input and 16-bit output.

---

4. `register16bit(clk, chipSelect, reset, regWrite, enable, inR, outR)` **(2 marks)**

a. This represents a **single 16-bit memory word**. Each register is made up of **16 D flip-flops**.

b. `inR[15:0]` is the 16-bit data that the user may want to write into the register.

c. `outR[15:0]` is the 16-bit data that the user would read from the register.

d. The `enable` input comes from the decoder output — it is high only for the register at the selected address.

e. You must use a **generate block** to instantiate the 16 D flip-flops. This is the key new Verilog construct for this lab.

---

5. `mux32to1(in0, in1, ..., in31, select, muxOut)` **(2 marks)**

a. This is a **32-to-1 multiplexer** with 16-bit wide inputs and output.

b. It selects one 16-bit word from 32 register outputs based on the **5-bit** `select` input.

c. `in0` through `in31` are the outputs of the 32 `register16bit` instances.

d. `select[4:0]` is the address given by the user — the memory location the user wants to read.

e. `muxOut[15:0]` is the 16-bit output of the selected memory location.

---

6. `ram32byte(clk, chipSelect, reset, outputEnable, writeEnable, address, writeData, memOut)` **(3 marks)**

a. This is the core RAM block. It contains **32 sixteen-bit registers**, giving 32 × 2 = 64 bytes of storage.

b. The RAM is **written synchronously** — only at the positive edge of the clock, and only if the chip is selected.

c. The RAM can be **read asynchronously** — at any time, as long as the chip is selected and `outputEnable` is high.

d. **Reset:** If `chipSelect` is high and `reset` is high, all bits in the RAM become 0 at the next positive clock edge.

e. **Write:** When `writeEnable` is high, `writeData[15:0]` is written into the register pointed to by the 5-bit `address[4:0]`.

f. **Read:** When `outputEnable` is high, `memOut[15:0]` reflects the 16-bit word stored at the location pointed to by `address[4:0]`. When `outputEnable` is low, `memOut` is driven to 0.

g. Wire together: `decoder5to32` → enables, 32 × `register16bit` → `mux32to1` → `mux2to1_16bits` → `memOut`.

---

7. `rom32byte(chipSelect, outputEnable, address, dataOut)` **(2 marks)**

a. This is a **32 × 16-bit Read-Only Memory**. It has no write port — its contents are fixed at design time.

b. The ROM has **no clock input** — all reads are purely combinational.

c. `address[4:0]` selects one of 32 memory locations.

d. `dataOut[15:0]` outputs the 16-bit word stored at the given address, but only when both `chipSelect` and `outputEnable` are high. Otherwise, `dataOut` is driven to 0.

e. You may implement the ROM contents using a `case` statement or an `initial` block with a register array. Pre-load contents as specified in the waveform/test outputs.

---

8. `ram1kb(clk, chipSelect, reset, outputEnable, writeEnable, address, writeData, memOut)` **(3 marks)**

a. This module stacks **32 instances of ram32byte** to form a larger RAM block.

b. The **upper 5 bits** of the 10-bit address (`address[9:5]`) are decoded by a `decoder5to32` to generate per-block chip selects — only one `ram32byte` block is active at a time.

c. The **lower 5 bits** (`address[4:0]`) are passed through to all blocks as the within-block address.

d. An output mux (`mux32to1` or equivalent) selects the active block's `memOut` as the final output.

e. The `chipSelect` received by each `ram32byte` is the AND of the top-level `chipSelect` and the block's decoded enable — only the selected block is active.

---

9. `rom1kb(chipSelect, outputEnable, address, dataOut)` **(2 marks)**

a. This module stacks **32 instances of rom32byte**, mirroring the structure of `ram1kb`.

b. It has **no clock or write ports** — it is entirely combinational.

c. Address decoding and output selection follow the same scheme as `ram1kb`: upper 5 bits select the block, lower 5 bits select the word within the block.

d. Only one block's output is active at a time; all others output 0.

---

10. `memory_system(addr, RD_n, WR_n, M_IO_n, data_out)` **(3 marks)**

a. This is the **top-level module**. It connects `ram1kb` and `rom1kb` to the processor's 20-bit address bus.

b. The control signals from the bus are **active-low**: `RD_n` (read), `WR_n` (write), `M_IO_n` (memory access). These must be inverted before driving the internal modules.

c. **Address decoding:** The upper bits of `addr[19:0]` determine which chip is selected. When `M_IO_n` is low (memory access), the appropriate chip select is asserted based on the address range:

Address Range	Chip Selected	Upper Address Bits
`0x00000 – 0x003FF`	`ram1kb`	`addr[19:10] = 10'h000`
`0x00400 – 0x007FF`	`rom1kb`	`addr[19:10] = 10'h001`
`All others`	`None`	—

d. `data_out[15:0]` should be driven by the active chip's output. If neither chip is selected, `data_out` should be 0.

e. The address passed to each chip is `addr[9:0]` (the lower 10 bits).

f. `clk` and `reset` are passed through to `ram1kb` only (ROM has no clock).

Circuit Diagrams for Each Module

Refer to the circuit diagram PDF distributed in class for the structural wiring of each module. The diagrams show the exact connections between sub-components for: `D_ff`, `mux2to1_16bits`, `decoder5to32`, `register16bit`, `mux32to1`, `ram32byte`, `ram1kb`, `rom32byte`, `rom1kb`, and `memory_system`.

Waveforms Generated

The **outputs** folder in the implementation directory contains the expected GTKWave waveform screenshots for each module. Use these to verify your implementation before submission. Expected waveforms are provided for:

- `mux2to1_16bits`
- `decoder5to32`
- `register16bit`

- `mux32to1`
- `ram32byte`
- `ram1kb`
- `rom32byte`
- `rom1kb`
- `memory_system`

Marks Summary

Module	Marks
D_ff (provided)	—
mux2to1_16bits	1
decoder5to32	2
register16bit	2
mux32to1	2
ram32byte	3
rom32byte	2
ram1kb	3
rom1kb	2
memory_system	3
Total	**20**